Progetto di Programmazione Web e Mobile

A.A. 2025/2026

Fast Food

Relazione tecnica

Studente: Diego Guerini
Matricola: 51418A
Data consegna: 15 Luglio 2026

Università degli Studi di Milano
Dipartimento di Informatica

Indice

1 Premessa **2**

2 Requisiti **2**
2.1 Macro-scenari 2
2.2 Dati iniziali 2

3 Struttura e presentazione del sito **3**
3.1 Pagine principali 3
3.2 Navigazione e UX (User Experience) 3

4 Backend e gestione dati **5**
4.1 API REST e Documentazione 5
4.2 Modelli Dati (Mongoose) 5
4.3 Sicurezza e Validazioni 5

5 Operazioni richieste **7**
5.1 Funzionalità di Base 7
5.2 Flusso Ordini e Ritiro in Sede 7

6 Scelte implementative **8**

7 Demo e immagini **9**
7.1 Setup per demo in locale 9
7.2 Schermate 9
7.2.1 Home di onboarding con accesso a login e registrazione 9
7.2.2 Motore di raccomandazione e preferenze 9
7.2.3 Ricerca ristoranti e piatti con filtri dedicati 12
7.2.4 Dettaglio ristorante con menu e selezione piatti 13
7.2.5 Dettaglio piatto con informazioni complete 13
7.2.6 Carrello e checkout 14
7.2.7 Ristoratore: sezione ordini 15
7.2.8 Transizione degli stati di un ordine 16
7.2.9 Ristoratore: statistiche e gestione menù 18

8 Informazioni generali e note finali **21**

1 Premessa

Questa relazione illustra le scelte progettuali, la struttura e le funzionalità dell'applicazione web **FastFood**, sviluppata come prova d'esame individuale per il corso di Programmazione Web e Mobile.

Il sistema è una piattaforma per l'ordinazione online dedicata a una catena di fast food, che mette in comunicazione due tipologie di utenti: i *clienti*, che possono navigare il menu ed effettuare ordini, e i *ristoratori*, che gestiscono le proprie anagrafiche, l'offerta del menu e le code di preparazione.

Il documento descrive i requisiti soddisfatti, la struttura dell'interfaccia frontend, le logiche del backend RESTful e le relative scelte implementative.

2 Requisiti

L'applicazione soddisfa tutte le operazioni base richieste dalla specifica di progetto per lo svolgimento singolo, coprendo quattro macro-scenari principali.

2.1 Macro-scenari

- **Gestione profilo utente:** Sistema di registrazione e login con scelta del ruolo (Cliente o Ristoratore). Prevede il salvataggio dei dati anagrafici, l'impostazione delle preferenze alimentari e dei metodi di pagamento, nonché la modifica o rimozione dell'account.
- **Gestione del Ristorante:** Area riservata ai ristoratori per gestire i dati del locale, con validazione dell'unicità della Partita IVA. Consente di comporre il menu clonando prodotti dal catalogo di sistema o creando piatti personalizzati, di cui è possibile gestire ingredienti, categorie e sconti a tempo.
- **Gestione degli Ordini e Ricerca:** Motore di ricerca che permette di filtrare i ristoranti (per nome e luogo) e i piatti (per tipologia, nome, prezzo massimo e ingredienti). L'utente autenticato può gestire il carrello, concludere il checkout e monitorare lo stato dell'ordine tramite uno storico.
- **Gestione delle consegne (Ritiro in sede):** L'applicativo gestisce le transizioni di stato dell'ordine (*Ordinato* $\rightarrow$ *In preparazione* $\rightarrow$ *Consegnato*). La modalità di ritiro in sede calcola dinamicamente il tempo di attesa stimato, basandosi sul numero di ordini attualmente in coda presso lo specifico ristorante.

2.2 Dati iniziali

Al primo avvio dell'applicazione, il database viene popolato tramite il parsing del file `JSON` fornito dalla traccia dell'esame.

Questi record costituiscono il catalogo base a disposizione dei ristoratori.

3 Struttura e presentazione del sito

L'interfaccia frontend è stata sviluppata sfruttando il framework Bootstrap 5, mantenendo una corretta separazione tra marcatura HTML5 e regole CSS3, garantendo nativamente un design *responsive* tramite un approccio *mobile-first*.

3.1 Pagine principali

Le viste dell'applicazione sono state suddivise per separare i contesti operativi:

- **Home e Menu (`index.html`):** Espone la navigazione globale fissata in alto (`sticky-top`) e un modulo di ricerca gestito tramite il componente `collapse` di Bootstrap. L'elenco dei piatti è renderizzato dinamicamente all'interno di un sistema a griglia.
- **Autenticazione (`login.html`, `register.html`):** Il form di registrazione adatta dinamicamente il DOM alle scelte dell'utente: selezionando il ruolo "Ristoratore", uno script disabilita e nasconde i campi relativi alle preferenze e ai metodi di pagamento, escludendoli così dal payload della richiesta HTTP.
- **Profilo Cliente (`profilo.html`):** Permette l'aggiornamento anagrafico. Il campo email è in sola lettura (attributo `disabled`) per prevenire mutazioni che invaliderebbero l'autenticazione.
- **Elenco Ristoranti (`ristoranti.html`):** Visualizza le anagrafiche dei locali partner. Cliccando su un ristorante, l'URL viene aggiornato apponendo l'ID del locale in *query string* (`?ristoratore=ID`); la Home intercetta questo parametro per filtrare dinamicamente i piatti.
- **Dashboard Ristoratore (`dashboard.html`):** Un'area amministrativa strutturata a schede (*tabs*): Ordini, Mio Menu, Impostazioni, Nuovo Piatto e Catalogo Base. Le operazioni CRUD sui piatti sono gestite tramite Modali Bootstrap per mantenere l'utente nello stesso contesto visivo.
- **Storico Ordini (`ordini.html`):** Riassume gli acquisti del cliente, impiegando classi CSS per colorare i badge in base all'avanzamento dell'ordine.

3.2 Navigazione e UX (User Experience)

L'interazione è governata da script che riducono i reload della pagina ottimizzando la UX:

- **Componenti Offcanvas:** L'accesso al carrello e al menu utente avviene tramite l'apertura laterale di componenti `Offcanvas`. Il menu personale viene generato a runtime valutando i dati utente salvati nel `localStorage`.
- **Skeleton Loaders:** Per ottimizzare l'esperienza utente durante le chiamate di rete (`fetch`), il DOM mostra temporaneamente delle card segnaposto (*skeleton*) che mantengono il layout della griglia fino all'arrivo dei dati JSON.

- **Validazione e notifiche Toast:** Durante la digitazione nei form, l'evento `input` innesca una validazione in tempo reale basata su espressioni regolari. L'esito delle operazioni (successo o errore) viene comunicato tramite *Toast* notificationali istanziati dinamicamente, che si auto-dismettono.
- **Animazioni computate:** Al click su "Aggiungi al carrello", il nodo dell'immagine viene clonato e traslato verso l'icona del carrello calcolando le coordinate a schermo tramite `getBoundingClientRect()`, sfruttando `requestAnimationFrame` per garantire fluidità.

4 Backend e gestione dati

Il backend è stato sviluppato in Node.js, impiegando Express.js per il routing delle API REST e Mongoose per l'interazione con il database MongoDB.

4.1 API REST e Documentazione

Il routing è organizzato in moduli per risorsa:

- **Meals:** Gestisce le operazioni sui piatti. In fase di creazione, il backend genera un `idMeal` custom basato su timestamp per distinguerli in modo univoco dai piatti standard inseriti al momento del *seeding*.
- **Restaurants:** Sulle chiamate `GET`, sfrutta il metodo `populate()` di Mongoose per risolvere le relazioni. Questo permette di restituire in un'unica risposta HTTP l'anagrafica del locale, i dati del titolare e il dettaglio base dei piatti del menu.
- **Orders:** Le statistiche del ristoratore (incassi, ordini pendenti, piatto più venduto) sono calcolate direttamente dal database tramite l'*aggregation pipeline* di MongoDB, evitando di sovraccaricare il backend Node.js con elaborazioni in memoria di grossi array.
- **Users:** Nelle rotte di fetch del profilo utente (`GET`), una proiezione a livello di query assicura che il campo password venga escluso dal payload di ritorno.

Tutti gli *endpoint* sono documentati nativamente tramite commenti JSDoc standardizzati per OpenAPI/Swagger.

4.2 Modelli Dati (Mongoose)

L'architettura documentale prevede quattro Schema:

- **User:** Utilizza il campo `ruolo` con un *enum* restrittivo (`['cliente', 'ristoratore']`) per definire i permessi di base e la tipologia di account.
- **Restaurant:** Conserva un riferimento `ObjectId` all'utente titolare e un array di riferimenti ai `Meal` offerti. Impone inoltre un indice univoco sul campo Partita IVA.
- **Meal:** Estende lo schema base fornito dalla traccia includendo metadati sul creatore e parametri per la gestione degli sconti (percentuale e data di scadenza).
- **Order:** Invece di salvare solo le referenze (`ObjectId`) dei piatti, l'ordine storicizza i dati essenziali come nome e prezzo al momento del checkout. Questo assicura che eventuali variazioni di listino future non corrompano i totali dello storico ordini.

4.3 Sicurezza e Validazioni

Per la gestione sicura delle credenziali ho utilizzato la libreria `bcrypt`. All'interno dello Schema `User`, un middleware Mongoose `pre('save')` intercetta le modifiche al campo

password ed esegue l'hashing prima del salvataggio nel database.

Per garantire una sicurezza architetturale robusta e non affidarsi unicamente ai controlli dell'interfaccia, è stato implementato un sistema di autorizzazione basato su **JSON Web Token (JWT)**. La trasformazione a JWT ha sostituito l'approccio iniziale basato su dati utente salvati in chiaro nel `localStorage` e sull'header custom `X-User-Role`.

Ora il backend emette un token firmato crittograficamente al momento del login e lo verifica su tutte le route protette, usando l'identità e il ruolo (*RBAC - Role Based Access Control*) contenuti nel payload del token per autorizzare operazioni sensibili su profilo, menu, ordini e ristorante. Se un attore non autorizzato tenta di interrogare le rotte amministrative direttamente (ad esempio tramite Postman), il server neutralizza la chiamata restituendo uno status code `403 Forbidden` o `401 Unauthorized`.

Lato frontend, ho centralizzato la gestione della sessione in un helper unico: l'applicazione legge ora il ruolo e l'ID decodificando il JWT, anziché affidarsi a un oggetto utente persistito in locale. Tutte le chiamate protette inviano il token tramite l'header `Authorization: Bearer <token>`. Il flusso di login e logout è stato riadattato a questa architettura, garantendo che le pagine principali funzionino in totale sicurezza senza esporre dati sensibili nel browser.

Infine, per prevenire vulnerabilità XSS (*Cross-Site Scripting*), ho implementato una funzione di utility che effettua l'*escaping* dei caratteri speciali prima di renderizzare gli input degli utenti nel DOM.

5 Operazioni richieste

L'applicazione implementa integralmente le funzionalità richieste per i progetti singoli.

5.1 Funzionalità di Base

- **Autenticazione Unificata:** Login cross-ruolo che autentica gli utenti verificando l'hash della password (via JWT) e li reindirizza in base ai loro permessi operativi.
- **Gestione Menu:** I ristoratori possono popolare la propria offerta copiando i piatti dal catalogo di sistema o creando nuove entità, di cui gestiscono liberamente caratteristiche e sconti.
- **Vincoli Logici del Carrello:** È stato implementato un blocco nella *business logic* del frontend che impedisce all'utente di mescolare, all'interno della stessa istanza del carrello, piatti appartenenti a ristoratori diversi.
- **Filtri Multidimensionali:** La ricerca permette di filtrare i ristoranti per nome e luogo, e i piatti per nome, categoria, prezzo massimo e ingredienti. Quest'ultimo controllo ispeziona direttamente l'array degli ingredienti associato al modello del piatto.

5.2 Flusso Ordini e Ritiro in Sede

Il ciclo di vita dell'ordine transita per gli stati obbligatori: *Ordinato*, *In preparazione*, *Consegnato*.

La logica per il ritiro in sede è calcolata a runtime dal backend in fase di `POST`: il controller effettua un `countDocuments` per individuare gli ordini del medesimo ristorante che non sono ancora stati consegnati. Il tempo di attesa stimato viene quindi calcolato sommando un tempo minimo di preparazione (10 minuti) a un incremento fisso (5 minuti) per ogni ordine attualmente in coda.

6 Scelte implementative

Le seguenti decisioni architetturali sono state adottate per ottimizzare l'efficienza e la manutenibilità del progetto:

- **Filtraggio In-Memory:** Per evitare eccessive chiamate HTTP durante la digitazione sui campi di ricerca, i dati vengono scaricati integralmente all'avvio della pagina. Il filtraggio avviene poi in memoria lato client utilizzando metodi nativi come `Array.prototype.filter()`, garantendo risultati istantanei nell'interfaccia.
- **Centralizzazione e Principio DRY:** Le funzioni di utilità trasversali, come il calcolo del prezzo scontato, sono centralizzate nel modulo `utils.js`. Questa funzione valuta dinamicamente la data corrente rispetto alla scadenza della promozione, permettendo di aggiornare i prezzi a schermo (nei menu e nel carrello) senza richiedere logica aggiuntiva al backend.
- **Promises per i Modali di Conferma:** Per sostituire la funzione nativa e bloccante `window.confirm()`, ho creato un modale personalizzato basato sulle `Promise`. Durante le operazioni critiche (come l'eliminazione di un piatto o del profilo), l'esecuzione viene sospesa tramite `async/await` in attesa dell'input dell'utente, mantenendo il browser fluido e reattivo.
- **Event Delegation:** In viste con elementi del DOM creati e distrutti frequentemente (come la lista degli ordini nella Dashboard o i bottoni del menu), ho adottato il pattern dell'*Event Delegation*. Invece di associare innumerevoli *event listener* ai singoli nodi, ho posizionato l'ascoltatore sull'elemento genitore, migliorando le prestazioni ed evitando problemi di *memory leak*.
- **Delegazione logica al Database (Aggregation Pipelines):** Invece di caricare e processare array massicci di dati sul server Node.js per elaborare le statistiche della dashboard, ho sfruttato le *Aggregation Pipelines* di MongoDB. Calcoli onerosi, come la somma degli incassi totali o l'identificazione del piatto più venduto, vengono eseguiti direttamente dal motore del database. Questo approccio riduce drasticamente sia la quantità di dati trasferita sulla rete, sia l'occupazione di memoria lato backend.

7 Demo e immagini

7.1 Setup per demo in locale

All'interno della cartella consegnata, nel file `README.md` sono presenti le istruzioni base per l'inizializzazione del progetto.

Per il corretto avvio dell'istanza backend, è strettamente necessario creare un file `.env` nella root del progetto, contenente le seguenti variabili di ambiente:

- `PORT`: (Opzionale) Indica la porta TCP su cui il web-server Node.js rimarrà in ascolto (es. `PORT=3000`). Se omessa, il server effettuerà il fallback automatico sulla porta 3000.
- `MONGO_URI`: La stringa di connessione fondamentale per l'accesso al cluster MongoDB Atlas.
- `JWT_SECRET`: La chiave segreta utilizzata dal backend per firmare e verificare i JSON Web Token.

Per facilitare le operazioni di testing ed evitare la necessità di registrare nuovi utenti, nella cartella principale è stato allegato un file `credentials.txt`. Questo documento contiene le credenziali in chiaro di due account pre-esistenti (un Ristoratore e un Cliente) già popolati con dati e ordini di prova.

Seguono immagini dimostrative.

7.2 Schermate

7.2.1 Home di onboarding con accesso a login e registrazione

Le interfacce di autenticazione validano gli input in tempo reale tramite espressioni regolari (Regex), fornendo un feedback visivo immediato. Il form di registrazione è reattivo: selezionando il ruolo "Ristoratore", il DOM si adatta dinamicamente disabilitando i campi superflui (preferenze e pagamenti) per escluderli dal payload HTTP.

7.2.2 Motore di raccomandazione e preferenze

L'applicazione include una vetrina dinamica denominata "Consigliati per te". Quando un cliente imposta una o più preferenze alimentari (es. "vegan, beef") all'interno del proprio profilo, il motore di raccomandazione filtra in tempo reale il catalogo globale. Il sistema identifica le corrispondenze effettuando un match tra i tag scelti dall'utente e i campi categoria o ingredienti dei piatti, popolando la cima della Home Page con una selezione su misura per il cliente.

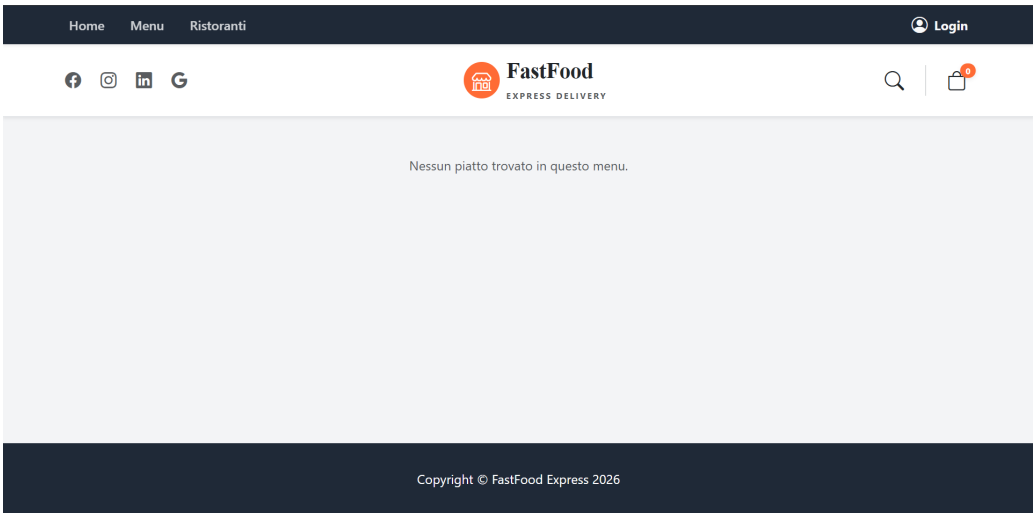


Figura 1: Homepage

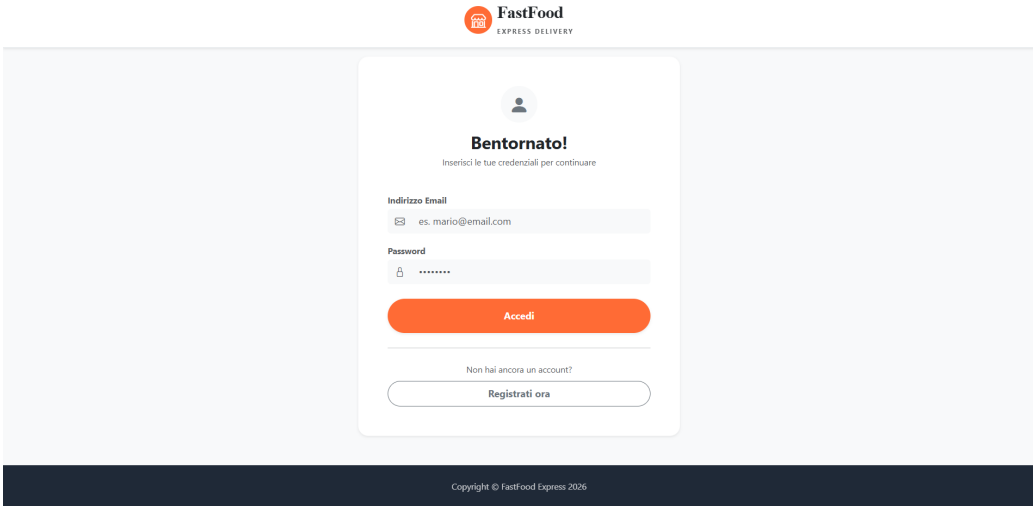


Figura 2: Pagina di login

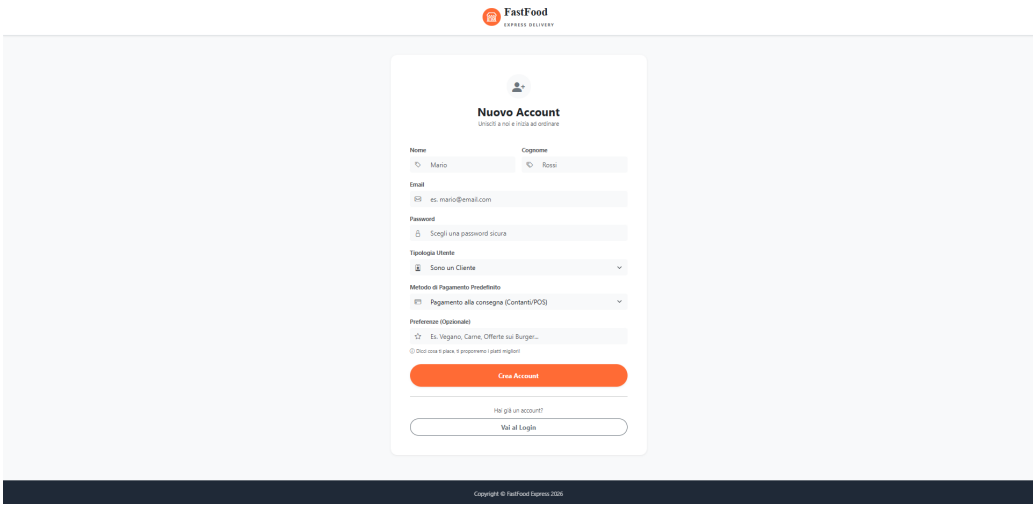


Figura 3: Pagina di registrazione

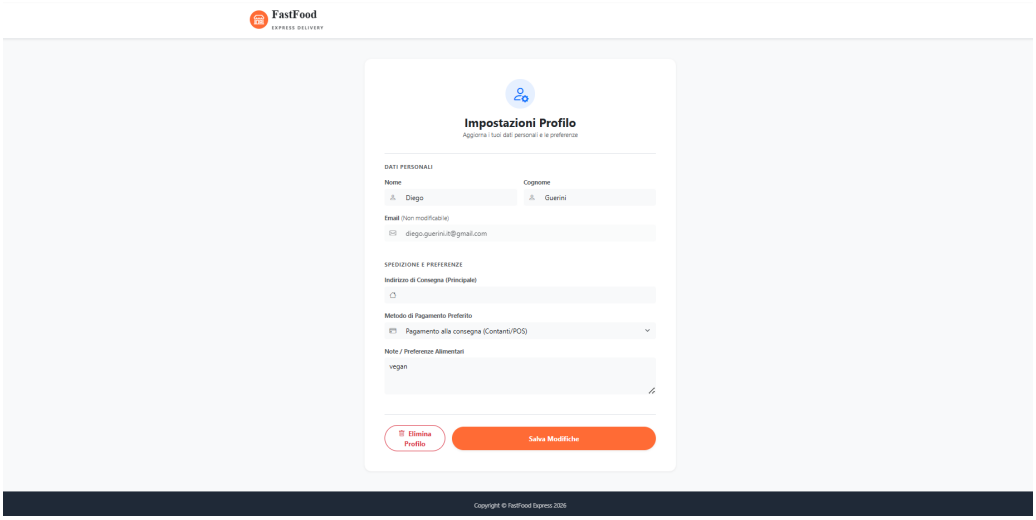


Figura 4: Profilo utente con opzioni di modifica ed eliminazione

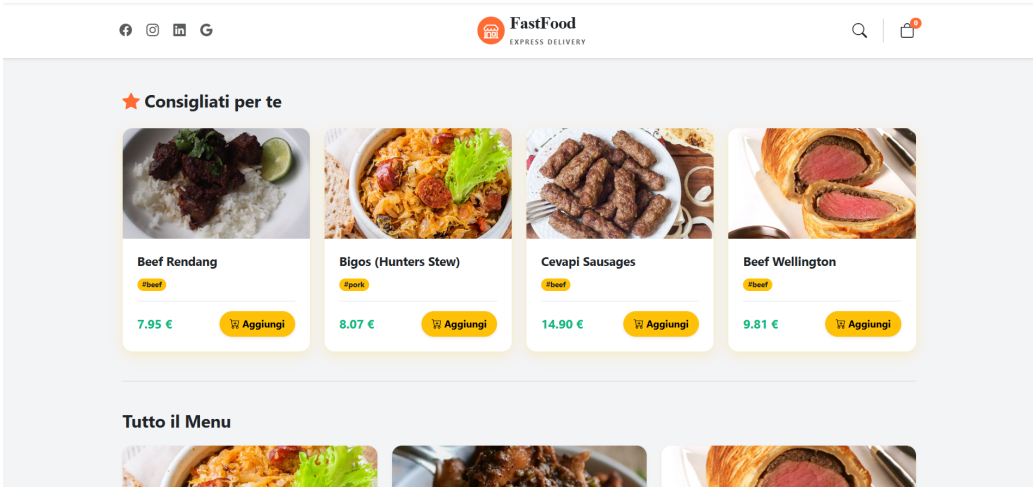


Figura 5: Sezione "Consigliati per te"

7.2.3 Ricerca ristoranti e piatti con filtri dedicati

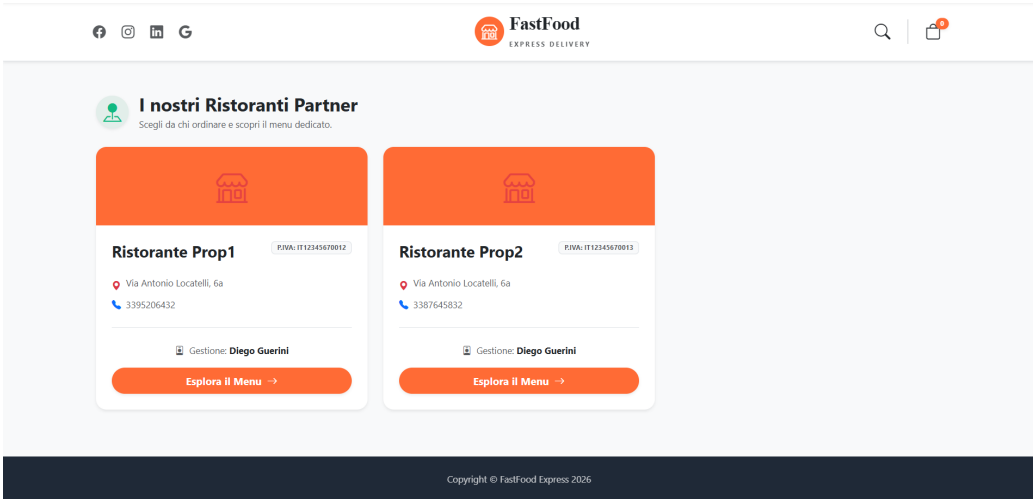


Figura 6: Ricerca per ristorante

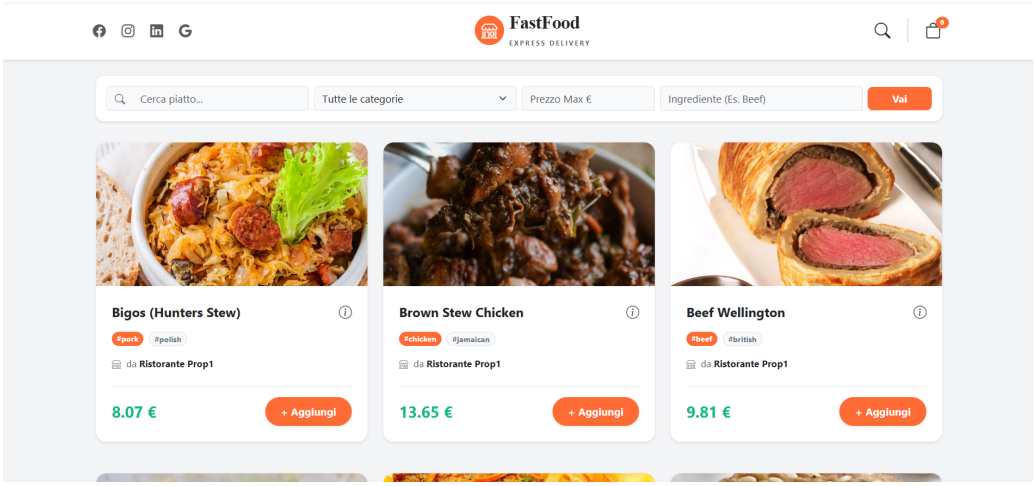


Figura 7: Ricerca per piatto

7.2.4 Dettaglio ristorante con menu e selezione piatti

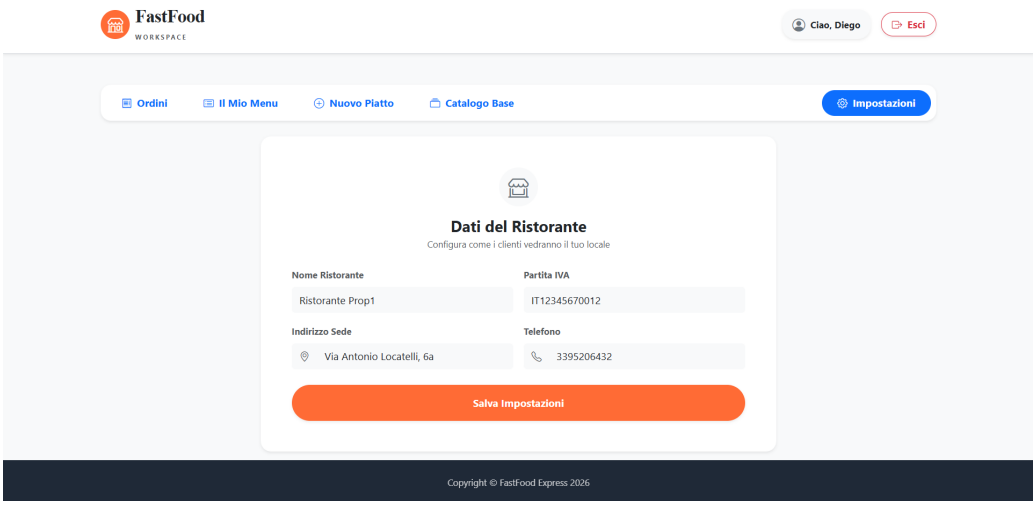


Figura 8: Pagina di dettaglio del ristorante

7.2.5 Dettaglio piatto con informazioni complete

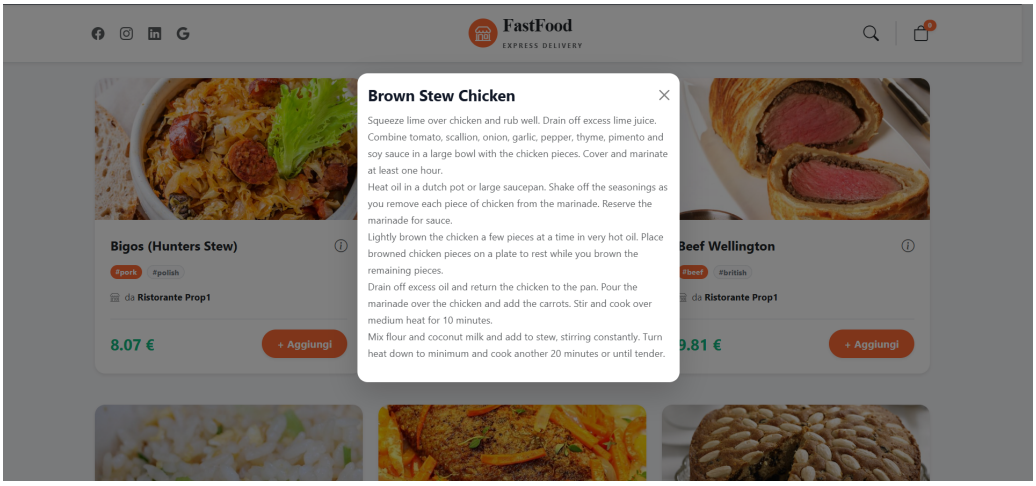


Figura 9: Dettaglio piatto

7.2.6 Carrello e checkout

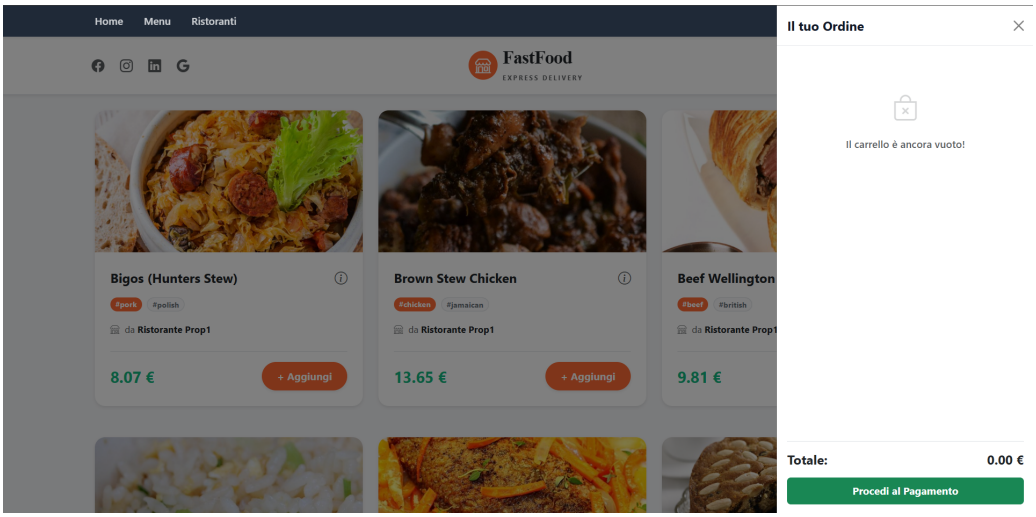


Figura 10: Carrello vuoto

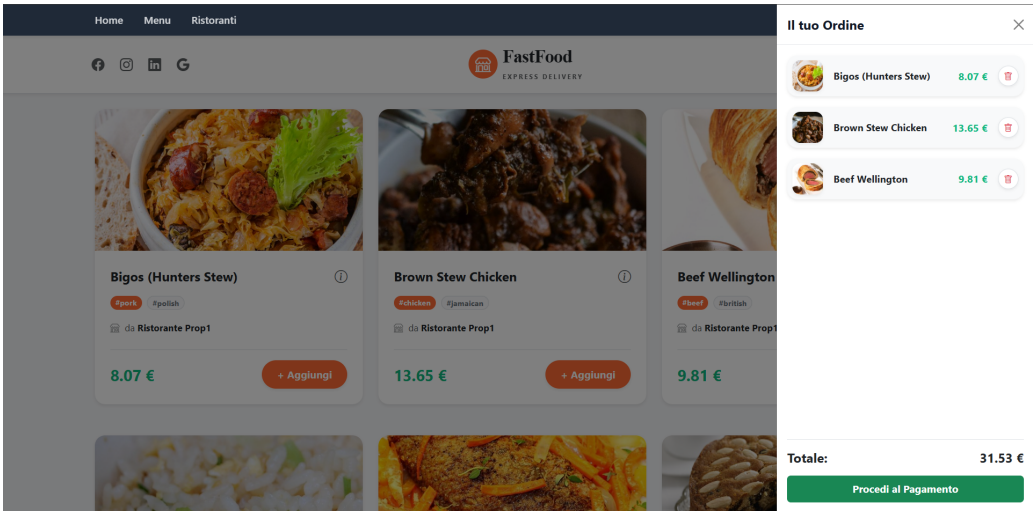


Figura 11: Carrello popolato

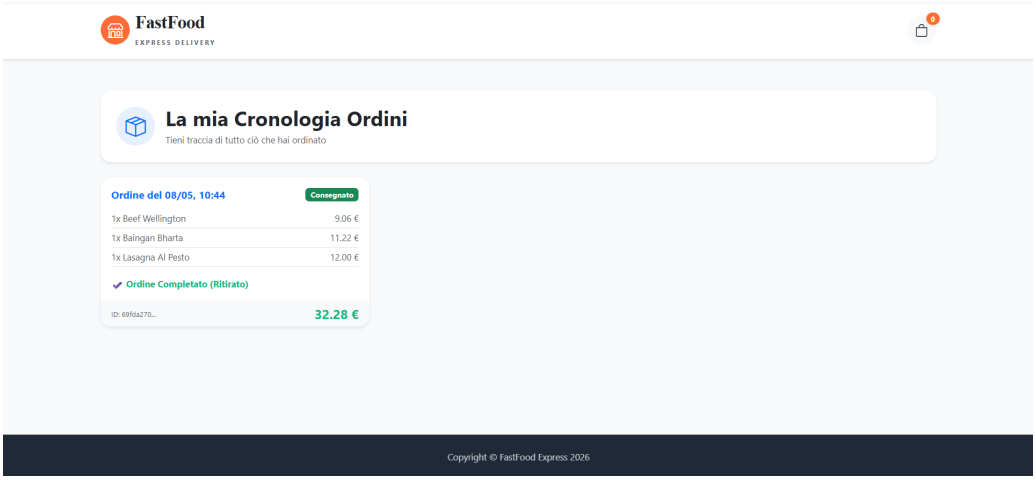


Figura 12: Storico ordini

7.2.7 Ristoratore: sezione ordini

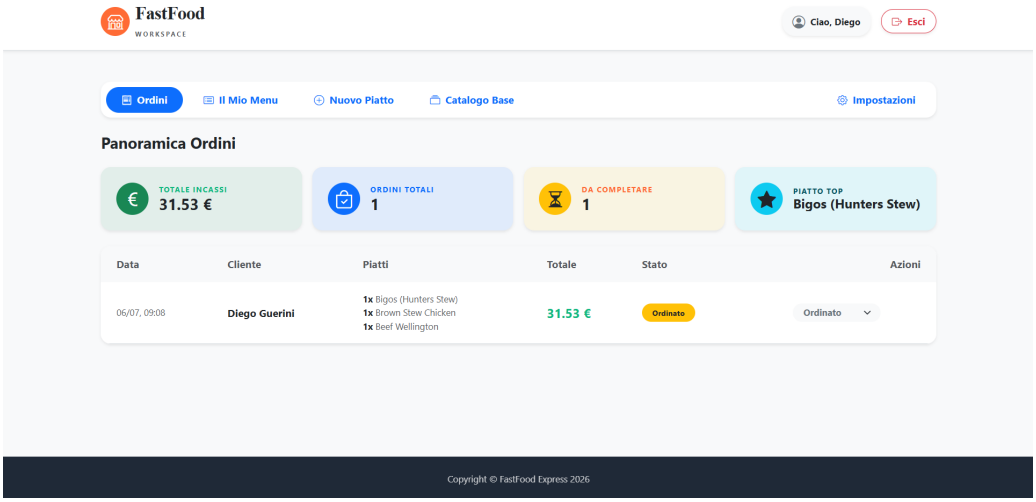


Figura 13: Pagina completa con ordini da prendere in carico dal ristorante

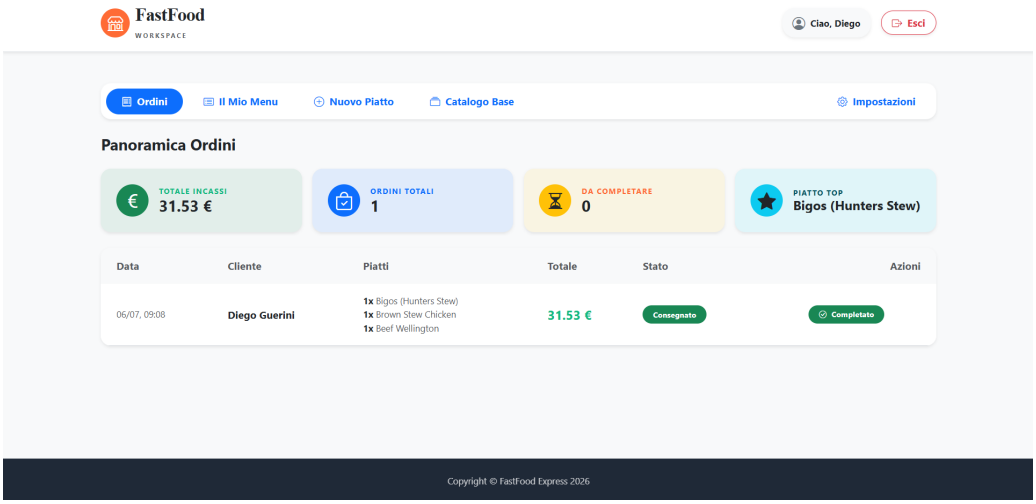


Figura 14: Dettaglio: coda ordini vuota

7.2.8 Transizione degli stati di un ordine

Dopo che il cliente, dalla pagina del carrello, invia l'ordine al ristorante, questo comparirà immediatamente nel suo Storico Ordini. In questa fase, il sistema espone all'utente un tempo di attesa stimato, calcolato dinamicamente in base al quantitativo di piatti richiesti nell'ordine.

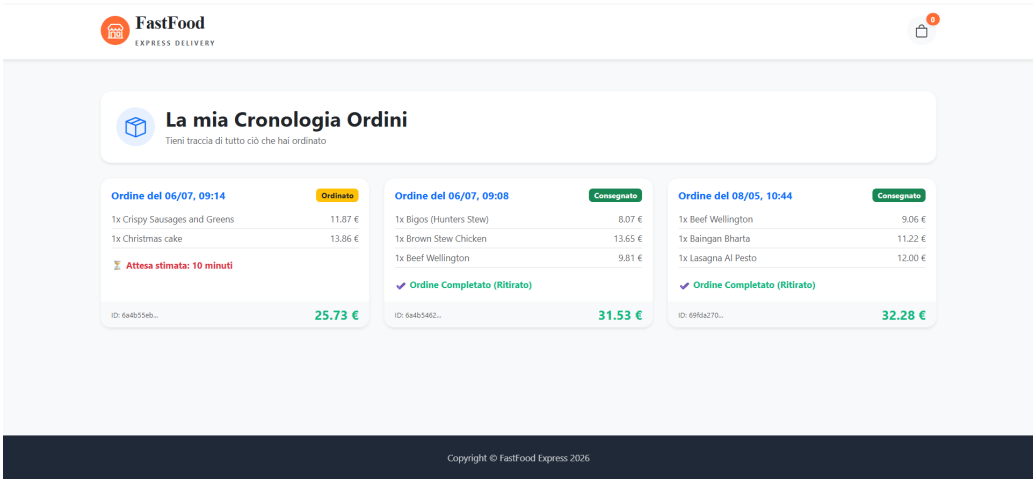


Figura 15: Lato cliente: nuovo ordine con tempo di attesa stimato

Lato ristoratore, il flusso di gestione è stato mantenuto essenziale e aderente alle specifiche della traccia, prevedendo l'avanzamento attraverso tre unici stati sequenziali: *Ordinato*, *In preparazione* e *Consegnato*.

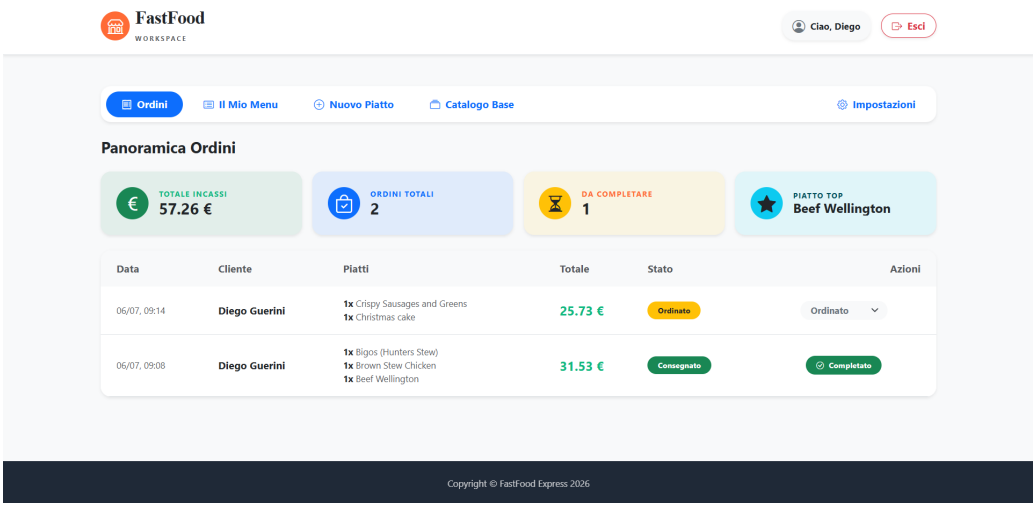


Figura 16: Dashboard ristoratore: ricezione del nuovo ordine (stato iniziale "Ordinato")

Il ristoratore prende in carico la comanda aggiornandone lo stato. Questa transizione si riflette lato client, fornendo così un riscontro sull'avanzamento della preparazione del proprio pasto.

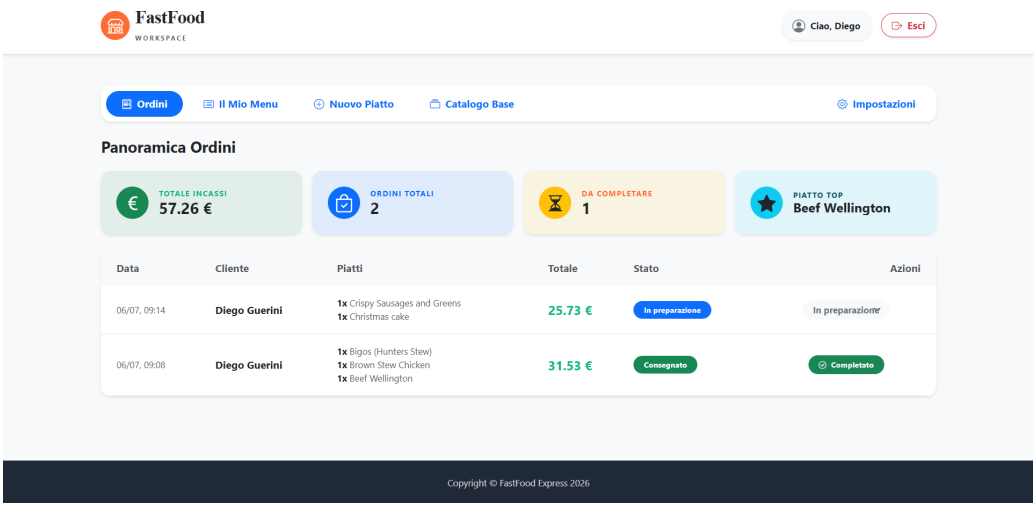


Figura 17: Dashboard ristoratore: transizione dell'ordine in "In preparazione"

Infine, una volta che il pasto è pronto per il ritiro in sede da parte del cliente, il ciclo di vita si conclude.

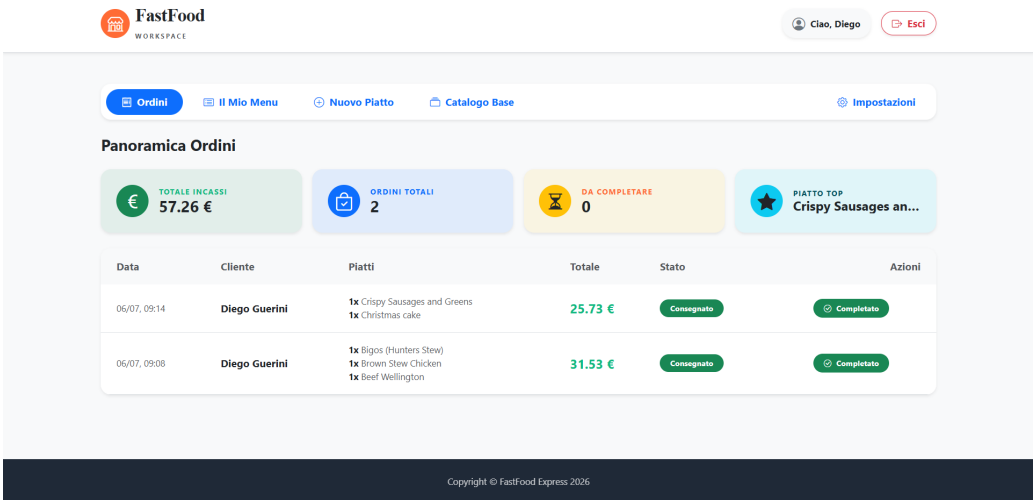


Figura 18: Dashboard ristoratore: conclusione dell'ordine (stato "Consegnato")

7.2.9 Ristoratore: statistiche e gestione menù

La Dashboard del ristoratore rappresenta il centro di controllo dell'attività. Nella vista principale (sezione Ordini) è possibile monitorare la coda delle preparazioni e, parallelamente, visualizzare le statistiche di vendita del ristorante calcolate dinamicamente dal database.

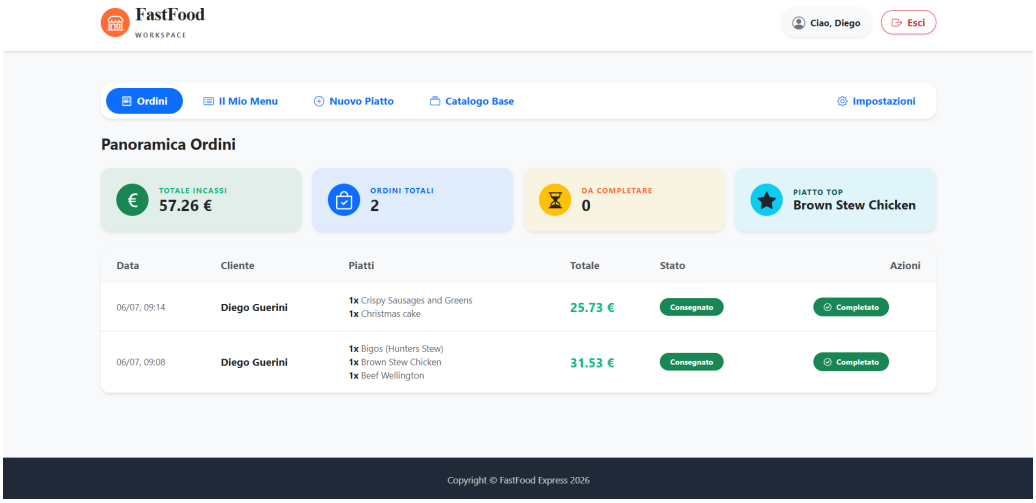


Figura 19: Dashboard principale: coda ordini e statistiche del ristorante

Spostandosi nella sezione "Il mio menu", il ristoratore ha accesso alla lista dei piatti attualmente offerti nel proprio locale, precedentemente selezionati dal catalogo globale o creati da zero.

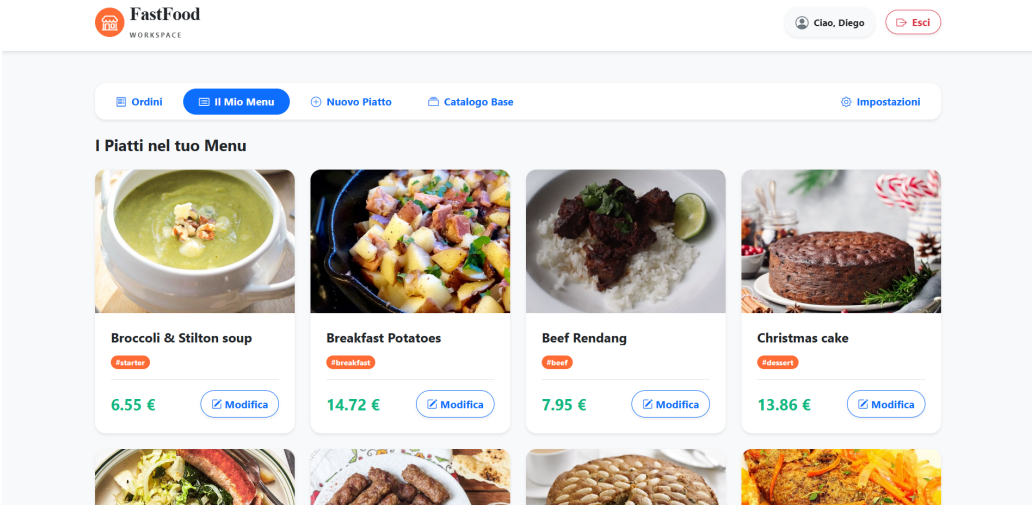


Figura 20: Sezione "Il mio menu": i piatti attivi nel ristorante

L'applicativo offre la massima flessibilità nella composizione dell'offerta: la scheda "Nuovo piatto" permette di creare pietanze completamente personalizzate che non sono presenti nel database standard.

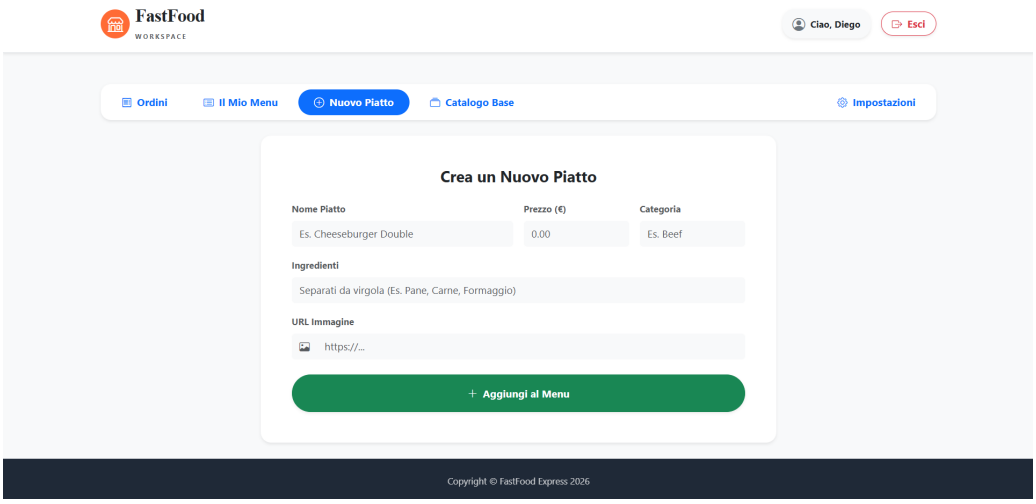


Figura 21: Sezione "Nuovo piatto": form per la creazione di piatti custom

In alternativa, tramite la sezione "Catalogo base", il ristoratore può scorrere l'intero database di sistema e aggiungere al proprio menù i piatti predefiniti in modo rapido.

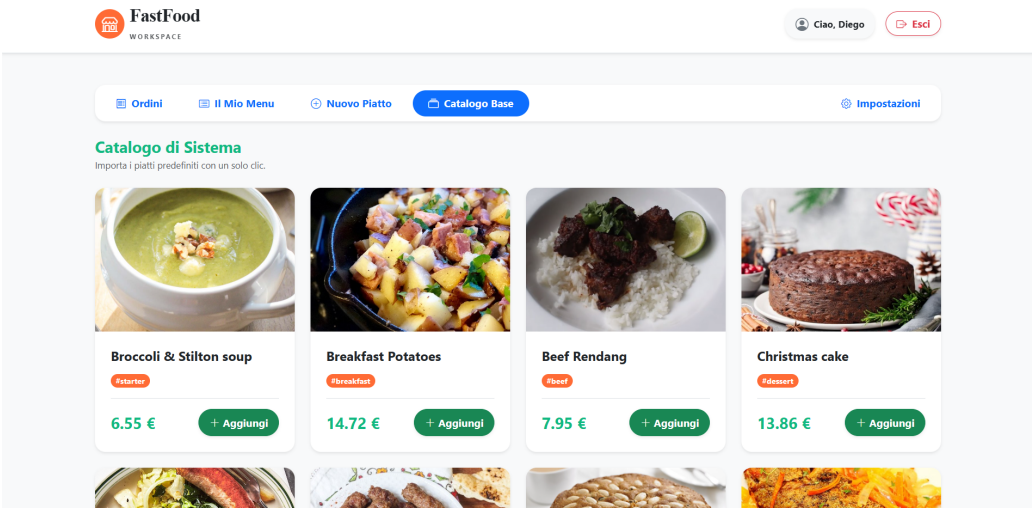


Figura 22: Sezione "Catalogo base": lista globale dei piatti di sistema

Infine, ogni piatto presente nel menù del ristorante può essere aggiornato tramite l'apposita schermata di modifica. Oltre a permettere l'alterazione dei campi anagrafici del prodotto (nome, prezzo, categoria, immagine e ingredienti), l'interfaccia include una funzionalità aggiuntiva per la gestione delle promozioni a tempo. Il ristoratore può infatti definire una percentuale di sconto personalizzata e impostare con precisione la data e l'ora in cui l'offerta terminerà automaticamente.

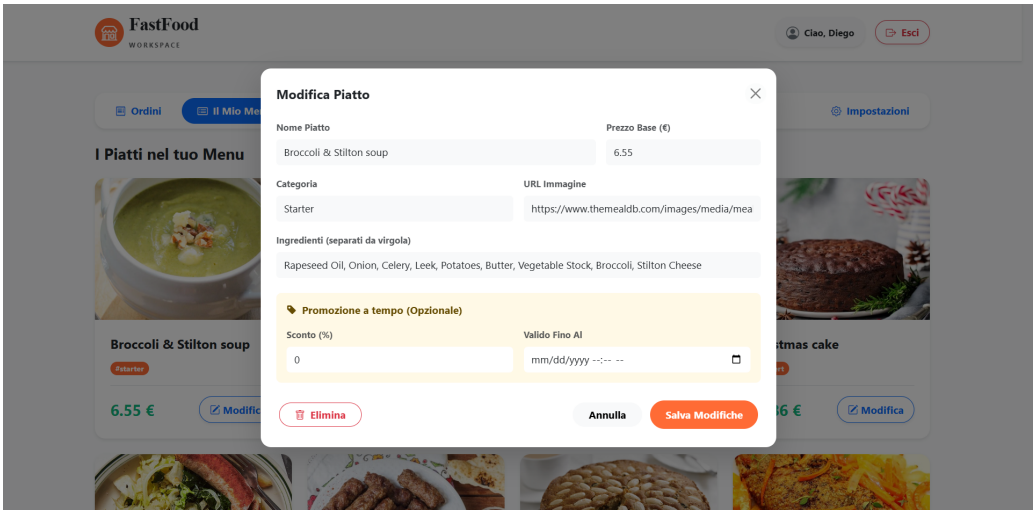


Figura 23: Modifica piatto: aggiornamento dettagli e impostazione promozioni a tempo

8 Informazioni generali e note finali

Il presente documento descrive l'applicativo **FastFood**, implementato secondo le tracce per l'esecuzione singola del progetto.

Con l'istanza server in esecuzione, l'applicativo è interrogabile sull'endpoint `http://localhost:[PORTA_SCELTA]`. La documentazione tecnica per le rotte backend (standard Swagger) è esposta alla route statica `/api/docs`. Il popolamento del layer di persistenza avviene a partire dal dataset iniziale `meal.json`, integrato in fase di bootstrap del database.